PQT MATLAB CODES

1. The two lines of regression are:
   $8x - 10y + 66 = 0$; $40x - 18y - 214 = 0$, and the variance of x is 9. Find
   i) Mean of x and y
   ii) Correlation coefficient between x and y.

```
%% AIM:
% To find the means (x̄, ȳ) and correlation coefficient (r) from two
% regression lines given the variance of x.

%% ALGORITHM:
% 1. Rewrite both regression equations in slope form
% 2. Identify which is y-on-x (byx) and which is x-on-y (bxy)
% 3. Compute r = sqrt(byx * bxy) (sign same as slopes)
% 4. Solve the two equations simultaneously to find (x̄, ȳ)
% 5. Display results

% Regression lines in slope form
% Line 1 (y on x): y = a1 + b1*x
% Line 2 (x on y): x = a2 + b2*y

% ========== REPLACE THESE 4 NUMBERS ==========
a1 = 30;      % intercept of y on x
b1 = -0.4;    % slope of y on x (byx)

a2 = 45;      % intercept of x on y
b2 = -0.5;    % slope of x on y (bxy)
% ============================================

% Given variance of x
var_x = 9;

% Find means (intersection of the two lines)
x_bar = (a2 + b2*a1) / (1 - b1*b2);
y_bar = a1 + b1*x_bar;

% Find correlation coefficient
r = sqrt(b1 * b2);
if b1 < 0
   r = -r;
end
```

```
% Results
fprintf('Mean of x = %.4f\n', x_bar);
fprintf('Mean of y = %.4f\n', y_bar);
fprintf('Correlation coefficient = %.4f\n', r);
```

2. Calculate the correlation co-efficient for the following heights (in inches) of fathers X their sons Y.

X:	65	66	67	67	68	69	70	72
Y:	67	68	65	68	72	72	69	71

```
%% AIM:
% To calculate the correlation coefficient between heights of fathers (X)
% and sons (Y) using both built-in function and manual calculation method,
% and to visualize the relationship with a scatter plot and regression line.

%% ALGORITHM:
% 1. Input the data arrays X (fathers' heights) and Y (sons' heights)
% 2. Calculate correlation coefficient using built-in corrcoef function
% 3. Manual calculation using Pearson correlation formula:
%    r = (n∑XY - ∑X∑Y) / sqrt([n∑X² - (∑X)²][n∑Y² - (∑Y)²])
%    where n = number of observations
% 4. Compute sums: ∑X, ∑Y, ∑XY, ∑X², ∑Y²
% 5. Calculate numerator and denominator using the formula
% 6. Display both results for verification
% 7. Plot scatter diagram of X vs Y
% 8. Calculate regression line coefficients:
%    slope (b) = r * (σy/σx)
%    intercept (a) = ȳ - b * x̄
% 9. Overlay regression line on scatter plot
% 10. Add labels, title, and grid to the plot

Code:
% Data
X = [65, 66, 67, 67, 68, 69, 70, 72];
Y = [67, 68, 65, 68, 72, 72, 69, 71];

% Method 1: Using built-in function
r_builtin = corrcoef(X, Y);
```

```
fprintf('Using corrcoef: r = %.4f\n', r_builtin(1,2));

% Method 2: Manual calculation
n = length(X);
sumX = sum(X);
sumY = sum(Y);
sumXY = sum(X.*Y);
sumX2 = sum(X.^2);
sumY2 = sum(Y.^2);

numerator = n*sumXY - sumX*sumY;
denominator = sqrt((n*sumX2 - sumX^2) * (n*sumY2 - sumY^2));
r_manual = numerator / denominator;

fprintf('Manual calculation: r = %.4f\n', r_manual);

% Scatter plot with regression line
figure;
scatter(X, Y, 'filled', 'b');
hold on;
% Regression line: Y = a + bX
b = r_manual * (std(Y)/std(X));
a = mean(Y) - b*mean(X);
x_vals = [min(X), max(X)];
y_vals = a + b*x_vals;
plot(x_vals, y_vals, 'r', 'LineWidth', 2);
xlabel('Fathers height (inches)');
ylabel('Sons height (inches)');
title(sprintf('Correlation coefficient r = %.4f', r_manual));
grid on;
```

3. A housewife buys the same cereal in successive weeks. If the buys cereal A, the next week she buys cereal B. However if she buys B or C, the next week she is 3 times as likely to buy A as the other cereal. In the long run how often she buys each of the 3 cereals?

```
%% AIM:
% To find the long-run stationary distribution (steady-state probabilities)
% of cereal purchases (A, B, C) by a housewife, given the transition
% probabilities between different cereals from week to week.


%% ALGORITHM:
% 1. Define the transition probability matrix P where P(i,j) represents
%    probability of moving from state i to state j in one step
%    States: 1 = Cereal A, 2 = Cereal B, 3 = Cereal C
%
% 2. Transition rules:
%    - From A: always goes to B (P(A→B)=1)
%    - From B: P(B→A)=0.75, P(B→C)=0.25, P(B→B)=0
%    - From C: P(C→A)=0.75, P(C→B)=0.25, P(C→C)=0
%
% 3. Set up equations for stationary distribution π = [π1, π2, π3]:
%    - πP = π (π times P equals π)
%    - π1 + π2 + π3 = 1 (probabilities sum to 1)
%
% 4. Rewrite πP = π as π(P - I) = 0
%    Transpose to get (P' - I)π' = 0
%
% 5. Form augmented matrix A = [P' - I; ones(1,3)] and b = [0;0;0;1]
%
% 6. Solve linear system A * π = b using matrix division (\)
% 7. Display stationary probabilities as fractions and percentages
% 8. Verify solution by checking πP = π and sum(π) = 1
```

```
Code:
% Transition matrix
P = [0, 1, 0;
    0.75, 0, 0.25;
    0.75, 0.25, 0];

% Solve pi * P = pi, sum pi = 1
A = P' - eye(3);
A = [A; ones(1,3)];
b = [0; 0; 0; 1];
pi = A \ b;

fprintf('Stationary distribution:\n');
fprintf('Cereal A: %.4f (%.2f%%)\n', pi(1), pi(1)*100);
fprintf('Cereal B: %.4f (%.2f%%)\n', pi(2), pi(2)*100);
fprintf('Cereal C: %.4f (%.2f%%)\n', pi(3), pi(3)*100);
```

4. Consider a Markov chain with 3 states and transition probability matrix

$$\begin{bmatrix} 0.6 & 0.2 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.6 & 0 & 0.4 \end{bmatrix}$$

```
%% AIM:
% To find the stationary probabilities (steady-state distribution) of a
% 3-state Markov chain given its transition probability matrix, and to
% verify the results using both analytical solution and power method.

%% ALGORITHM:
% 1. Define the transition probability matrix P of size 3×3, where
%    P(i,j) = probability of moving from state i to state j in one step
%
% 2. The stationary distribution π = [π1, π2, π3] satisfies:
%    (a) πP = π  (balance equations)
%    (b) π1 + π2 + π3 = 1  (normalization condition)
%
% 3. Rewrite πP = π as π(P - I) = 0
```

```
%    Take transpose: (P' - I)π' = 0
%
% 4. Form augmented matrix A by adding normalization equation:
%    A = [P' - I; ones(1,3)]
%    b = [0; 0; 0; 1]
%
% 5. Solve linear system A * π = b using matrix division (\)
%
% 6. Display stationary probabilities
%
% 7. Verify using power method:
%    - Raise P to a high power (P^100)
%    - Any row of P^n converges to π for large n
%    - Compare with analytical solution
```

Code:

```
        P = [0.6 0.2 0.2;
             0.1 0.8 0.1;
             0.6 0.0 0.4];

% Method: Solve πP = π with sum(π)=1
A = P' - eye(3);
A = [A; ones(1,3)];
b = [0; 0; 0; 1];
pi = A \ b;

fprintf('Stationary probabilities:\n');
fprintf('π1 = %.4f, π2 = %.4f, π3 = %.4f\n', pi(1), pi(2), pi(3));

% Verification using power method
P_power = P^100;
fprintf('\nVerification (P^100):\n');
fprintf('π1 = %.4f, π2 = %.4f, π3 = %.4f\n', P_power(1,1), P_power(1,2), P_power(1,3));
```

5. If customers arrive at a counter in accordance with a Poisson process with a mean rate of 3 per minute, find the probability that the interval between 2 consecutive arrivals is

   (i) more than 1 minute (ii) between 1 minute and 2 minutes

   (iii) 4 minutes or less

```
%% AIM:
% To compute probabilities of inter-arrival times in a Poisson process
% with mean arrival rate λ = 3 per minute. The inter-arrival times follow
% an exponential distribution with parameter λ.

%% ALGORITHM:
% 1. Define the arrival rate λ = 3 arrivals per minute
% 2. For a Poisson process, inter-arrival times T follow exponential
%    distribution with CDF: P(T ≤ t) = 1 - e^(-λt)
%    and survival function: P(T > t) = e^(-λt)
% 3. Calculate probability that interval exceeds 1 minute:
%    P(T > 1) = e^(-λ × 1)
% 4. Calculate probability that interval is between 1 and 2 minutes:
%    P(1 < T ≤ 2) = P(T > 1) - P(T > 2) = e^(-λ×1) - e^(-λ×2)
% 5. Calculate probability that interval is 4 minutes or less:
%    P(T ≤ 4) = 1 - e^(-λ×4)
% 6. Display all probabilities with 6 decimal places
% 7. (Optional) Plot the exponential distribution PDF and CDF

Code:
% Poisson process with lambda = 3 per minute
lambda = 3;

% (i) More than 1 minute
P_more_than_1 = exp(-lambda * 1);
fprintf('(i) P(interval > 1 minute) = %.6f\n', P_more_than_1);
```

```
% (ii) Between 1 and 2 minutes
P_between_1_and_2 = exp(-lambda * 1) - exp(-lambda * 2);
fprintf('(ii) P(1 < interval ≤ 2 minutes) = %.6f\n', P_between_1_and_2);

% (iii) 4 minutes or less
P_less_than_4 = 1 - exp(-lambda * 4);
fprintf('(iii) P(interval ≤ 4 minutes) = %.6f\n', P_less_than_4);
```

6. Customers arrived at a watch repair shop according to Poisson process at a rate of 1 per every 10 minutes and the service time is exponential random variable with mean 8 minutes,
    i) Find the average number of customers in the shop.
    ii) Find the average time a customer spends in the shop.
    iii) Find the average number of customers in the queue.
    iv) What is the probability that the server is idle?

```
%% AIM:
% To analyze the M/M/1 queueing system of a watch repair shop where
% customers arrive according to Poisson process and service times follow
% exponential distribution. Calculate performance measures including
% average number of customers, average time spent, queue length,
% and server idle probability.

%% ALGORITHM:
% 1. Define arrival rate (λ) and service rate (μ):
%    - Customers arrive every 10 minutes → λ = 1/10 = 0.1 customers/minute
%    - Service time averages 8 minutes → μ = 1/8 = 0.125 customers/minute
%
% 2. Calculate traffic intensity (utilization factor):
%    ρ = λ/μ
%    Condition for stable system: ρ < 1
%
% 3. Calculate performance measures using M/M/1 formulas:
%    a) Average number of customers in system (L) = ρ/(1-ρ)
%    b) Average time in system (W) = 1/(μ-λ)
```

```
%    c) Average number in queue (Lq) = ρ²/(1-ρ)
%    d) Probability server is idle (P0) = 1-ρ
%
% 4. Display all results with appropriate formatting
%
% 5. (Optional) Verify relationships using Little's Law:
%    L = λ × W
%    Lq = λ × Wq where Wq = W - 1/μ

Code:
% M/M/1 queue
lambda = 1/10;   % customers per minute (1 every 10 min)
mu = 1/8;        % service rate (mean 8 min -> 0.125 per min)

rho = lambda / mu;

% 1. Average number in system
L = rho / (1 - rho);

% 2. Average time in system (minutes)
W = 1 / (mu - lambda);

% 3. Average number in queue
Lq = rho^2 / (1 - rho);

% 4. Probability server is idle
P_idle = 1 - rho;

fprintf('1. Average number of customers in shop: %.4f\n', L);
fprintf('2. Average time a customer spends in shop: %.4f minutes\n', W);
fprintf('3. Average number of customers in queue: %.4f\n', Lq);
fprintf('4. Probability that server is idle: %.4f (%.1f%%)\n', P_idle, P_idle*100);
```

7. At a port there are 6 unloading berths and 4 unloading crews. When all the berths are full, arriving ships are diverted to an over flow facility 20 kms down the river. Tankers arrive according to a Poisson process with a mean of 1 every 2 h. It takes

for an unloading crew, on the average, 10 h to unload a tanker, the unloading time following an exponential distribution. Find

i) how many tankers are at the port on the average?

ii) how long does a tanker spend at the port on the average?

iii) what is the average arrival rate at the overflow facility?

```
%% AIM:
% To analyze the M/M/c/c queueing system (Erlang-B loss system) at a port
% with 4 unloading crews/berths. Tankers arrive according to Poisson process,
% and if all berths are busy, arriving tankers are diverted to an overflow
% facility. Calculate average number of tankers at port, average time spent,
% and overflow rate.
%% ALGORITHM:
% 1. Define system parameters:
%    - Arrival rate: λ = 0.5 tankers/hour (1 every 2 hours)
%    - Service rate: μ = 0.1 tankers/hour (10 hours per tanker)
%    - Number of servers: c = 4 crews/berths
% 2. Calculate offered load in Erlangs:
%    a = λ/μ = 5 Erlangs
% 3. Compute blocking probability using Erlang-B formula:
%    P_block = (a^c / c!) / Σ(a^k / k!) for k = 0 to c
% 4. Calculate performance measures:
%    (i) Average number of tankers at port:
%       L = a × (1 - P_block)
%    (ii) Average time a tanker spends at port:
%       λ_eff = λ × (1 - P_block)  [effective arrival rate]
%       W = L / λ_eff
%    (iii) Average arrival rate at overflow facility:
%       λ_overflow = λ × P_block
% 5. Display results with appropriate units (hours, minutes, tankers/day)


% M/M/c/c queue (Erlang-B)
lambda = 0.5;    % tankers per hour
```

```
mu = 0.1;       % service rate per crew
c = 4;          % number of crews (and berths in use)
a = lambda / mu; % offered load in Erlangs

% Compute blocking probability (Erlang-B)
num = (a^c) / factorial(c);
denom = 0;
for k = 0:c
    denom = denom + (a^k) / factorial(k);
end
P_block = num / denom;

% (i) Average number of tankers at port = average number in system (L)
% For M/M/c/c, L = a * (1 - P_block)
L = a * (1 - P_block);

% (ii) Average time a tanker spends at port (W) = L / lambda_eff
% lambda_eff = arrival rate that actually enter = lambda * (1 - P_block)
lambda_eff = lambda * (1 - P_block);
W = L / lambda_eff;  % hours

% (iii) Average arrival rate at overflow facility = lambda * P_block
lambda_overflow = lambda * P_block;

fprintf('(i) Average number of tankers at port: %.4f\n', L);
fprintf('(ii) Average time at port: %.4f hours (%.2f minutes)\n', W, W*60);
fprintf('(iii) Average arrival rate at overflow facility: %.4f tankers/hour\n',lambda_overflow);
fprintf('    (%.2f tankers per day)\n', lambda_overflow * 24);
```

8. There are 3 typists in an office. Each typist can type an average of 6 letters per hour. If letters arrive for being typed at the rate of 15 letters per hour

i) What fraction of time will all the typists be busy?

ii) Find the average number of letters waiting to be typed.

iii) What is the average time a letter has to spend for waiting and being typed?

```
%% AIM:
% To analyze the M/M/c queueing system for a typing pool with 3 typists.
% Letters arrive according to Poisson process at rate 15 per hour, and
% each typist can type 6 letters per hour (exponential service time).
% Calculate: (i) fraction of time all typists are busy, (ii) average
% number of letters waiting, (iii) average total time per letter.

%% ALGORITHM:
% 1. Define system parameters:
%    - Arrival rate: λ = 15 letters/hour
%    - Service rate per typist: μ = 6 letters/hour
%    - Number of servers: c = 3 typists
%
% 2. Calculate server utilization (traffic intensity):
%    ρ = λ/(c × μ)
%    Condition for stability: ρ < 1
%
% 3. Calculate P0 (probability that system is empty) using formula:
%    P0 = 1 / [ Σ_{n=0}^{c-1} (λ/μ)^n/n! + (λ/μ)^c/(c! × (1-ρ)) ]
%
% 4. Calculate performance measures:
%    (i) P(all typists busy) = P(≥ c in system)
%        = ( (λ/μ)^c × P0 ) / (c! × (1-ρ))
%
%    (ii) Average number of letters waiting (Lq):
%        Lq = (P0 × (λ/μ)^c × ρ) / (c! × (1-ρ)^2)
%
%    (iii) Average total time per letter (W):
%        L = Lq + λ/μ  [average number in system]
%        W = L/λ  [Little's Law]
%
% 5. Display results with appropriate units (hours and minutes)
```

Code:

```
% Queuing Theory: M/M/c model
lambda = 15;   % arrival rate (letters per hour)
mu = 6;       % service rate per typist (letters per hour)
c = 3;        % number of typists

rho = lambda / (c * mu);  % server utilization
```

```
% Calculate P0 (probability that system is empty)
sum1 = 0;
for n = 0:c-1
   sum1 = sum1 + (lambda/mu)^n / factorial(n);
end
sum2 = (lambda/mu)^c / (factorial(c) * (1 - rho));
P0 = 1 / (sum1 + sum2);

%% (i) Fraction of time all typists are busy
% All busy = probability that at least c letters are in system
% P_busy = sum_{n=c}^\infty P_n = ( (lambda/mu)^c * P0 ) / (factorial(c) * (1 - rho))
P_all_busy = ((lambda/mu)^c * P0) / (factorial(c) * (1 - rho));
fprintf('(i) Fraction of time all typists are busy: %.4f\n', P_all_busy);

%% (ii) Average number of letters waiting to be typed (Lq)
Lq = (P0 * (lambda/mu)^c * rho) / (factorial(c) * (1 - rho)^2);
fprintf('(ii) Average number of letters waiting: %.4f letters\n', Lq);

%% (iii) Average time a letter spends waiting + being typed (W)
% First, average number in system L = Lq + lambda/mu
L = Lq + lambda/mu;
% Little's Law: W = L / lambda
W = L / lambda;
fprintf('(iii) Average total time per letter: %.4f hours (%.2f minutes)\n', W, W*60);
```